INSAlgo



Data Structures

Structures de données
Une première définition

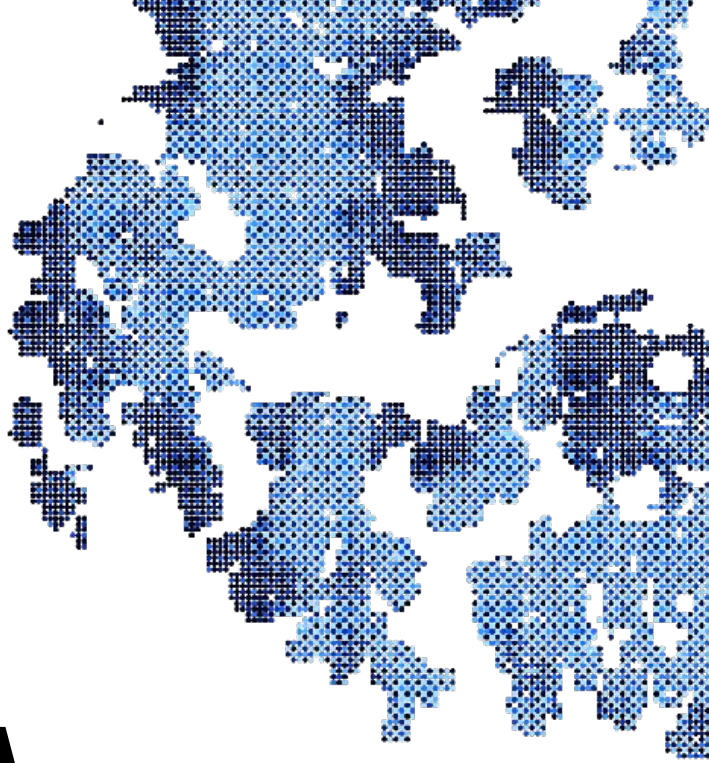
Wikipedia (FR) : une manière d'organiser et stocker les données pour les traiter plus facilement

- Une abstraction de la donnée qui est facile à manipuler
- Contient plus que les données elle-mêmes : les données sont organisées d'une certaine façon
- Un ensemble d'opérations sont associées à cette organisation

→ Important de savoir quelles structures de données existent, mais également quelles sont les opérations qui existent pour les manipuler



INSAlgo



Dynamic Array

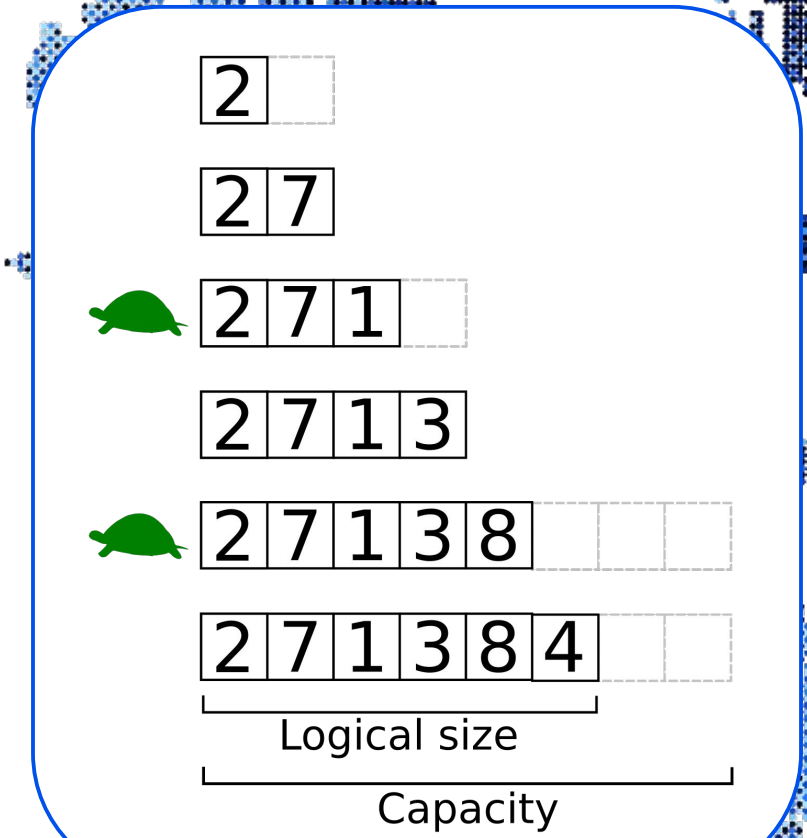
Aussi appelé vecteur/vector, c’est la “liste” en python

Dynamic Arrays

Objectif : accéder rapidement à un élément d'un "conteneur" et y ajouter d'autres éléments

Implémentation

- Le conteneur prend plus de place que nécessaire
- Tant que cette espace supplémentaire n'est pas rempli, l'ajout d'un élément ne coûte rien
- Quand il n'y a plus d'espace supplémentaire libre, on recopie le conteneur avec davantage d'espace libre



Dynamic Arrays

Quelle est la complexité pour ajouter un élément ?

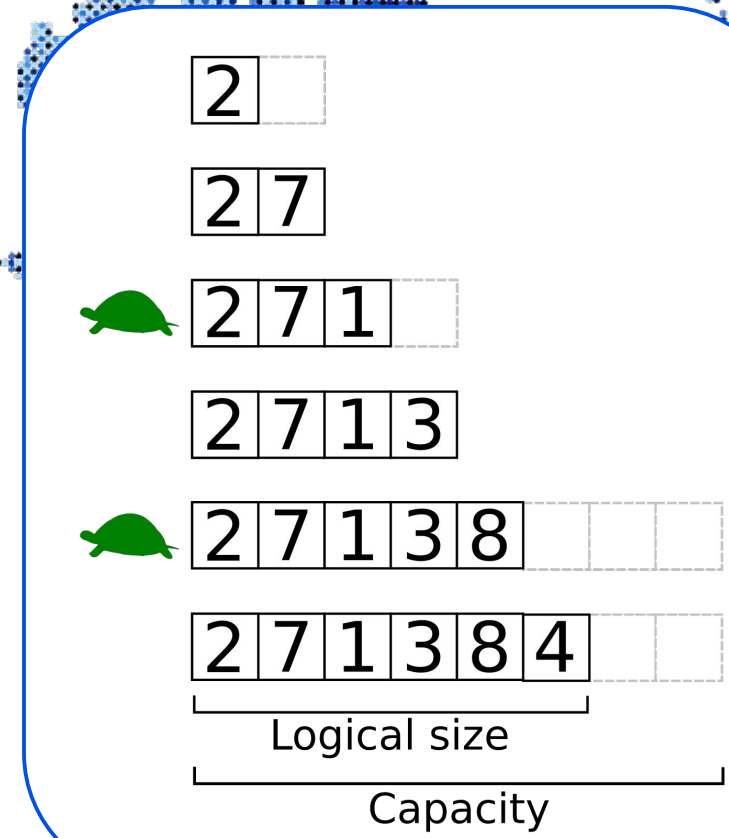
- Si la capacité c1 est atteinte, alors on étend à la capacité c2 = 2 * c1, le coût de cette opération étant c1
- Ensuite le temps pour ajouter sera constant pour encore c1 insertions

→ Dans le cas général, ajouter coûte O(1)

Quelle est la complexité pour accéder à un élément ?

Il suffit de se souvenir de l'adresse du premier élément, auquel on ajoute l'index de l'élément auquel on veut accéder

→ O(1)



Dynamic Arrays
Les opérations fréquentes

Opération	Python	Complexité
Accéder à l'élément en position *i*	arr[i]	O(1)
Ajouter *v* à la fin	arr.append(v)	O(1) en moyenne O(n) dans le pire cas
Insérer *v* à la position *i*	arr.insert(i, v)	O(n)
Trouver la position de *v*	arr.index(v)	O(n)

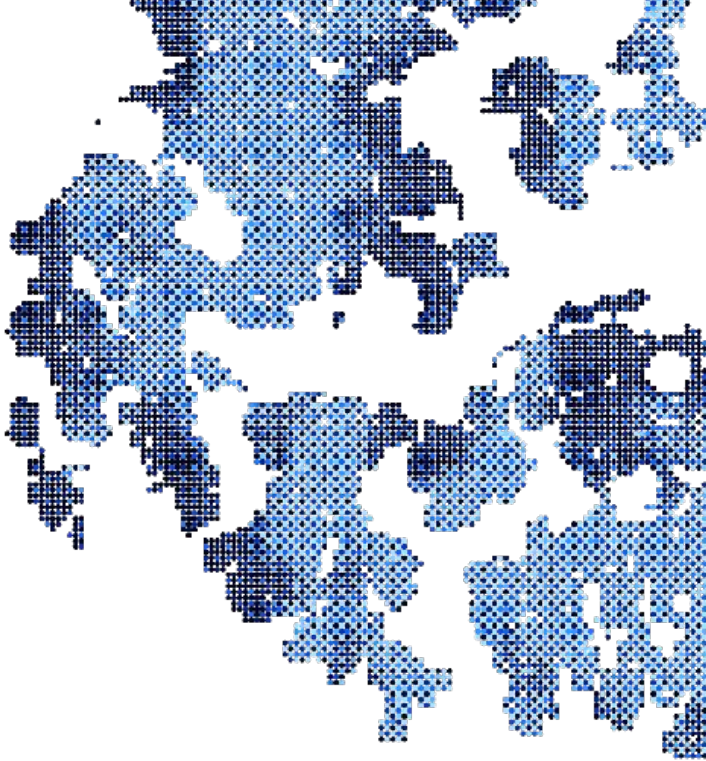
Quand l'utiliser ?

Il y a beaucoup de cas d'usage possibles ; pensez à utiliser une autre structure de données lorsque vous utilisez beaucoup d'opération qui ne sont pas en O(1)

Dynamic Arrays - Cheat sheet

Opération	**Python**	**Complexité**
Créer une liste	L = list() ou L = []	O(1)
Accéder à l'élément en position *i*	arr[i]	O(1)
Ajouter *v* à la fin	arr.append(i)	O(1) avg / O(n) max
Insérer *v* à la position *i*	arr.insert(i, v)	O(n)
Trouver la position d'un élément *v*	arr.index(v)	O(n)
Enlever le premier élément valant *x*	arr.remove(x)	O(n)
Enlever l'élément à la position *i*	arr.pop(i)	O(n)
Vider la liste	arr.clear()	O(n)
Récupérer la longueur de la liste	arr.len()	O(1)
Trier la liste	arr.sort(key=function)	O(n*log(n))
Compter le nombre d'occurrence de *x*	arr.count(x)	O(n)

INSAlgo



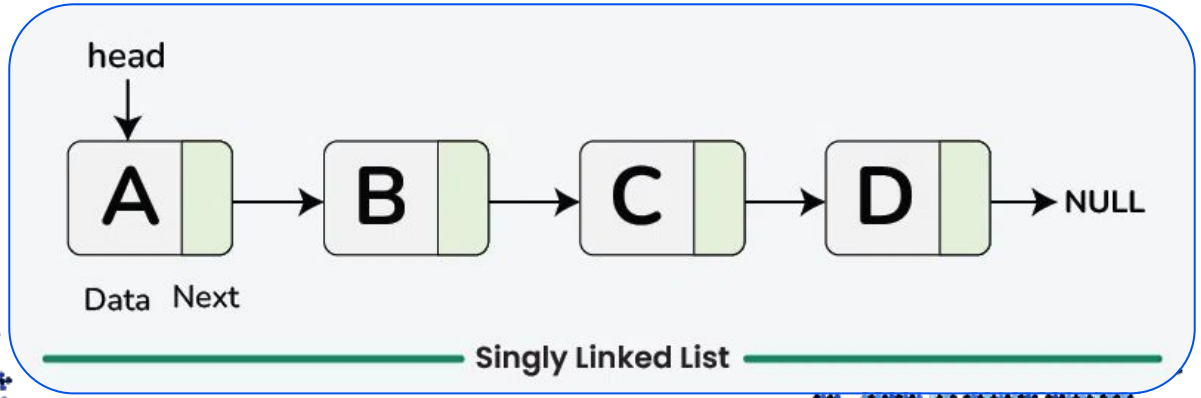
Linked list

Liste chaînée

Liste chaînée
Définition

Une liste chaînée est une séquence de noeuds comprenant 2 éléments :

- Une donnée (dans l'exemple : un caractère)
- Un lien vers le prochain noeud (le dernier *lien* étant NULL pour signifier la fin de la liste chaînée)



Liste chaînée
Avantages et inconvénients

Avantages :

- Insertion et suppression en O(1) si on possède le noeud concerné
- Pas de réallocation mémoire lors d'ajout, un noeud créé à la fois

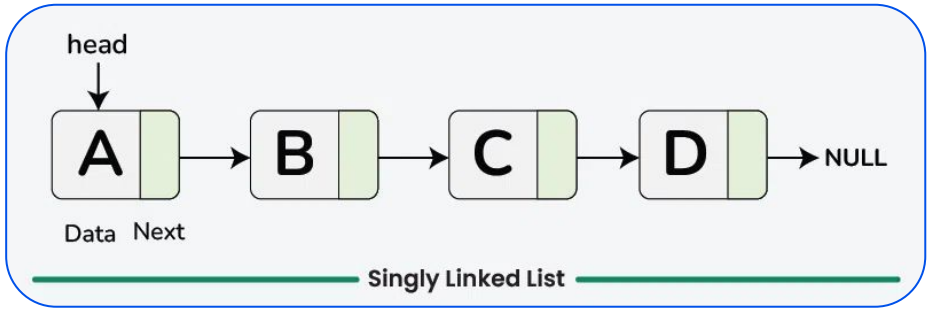
Inconvénients :

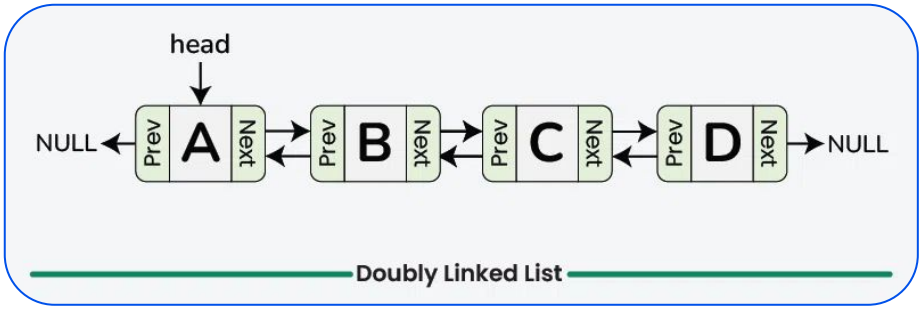
- Accès aléatoire en O(n), il n'existe pas de list[i]
- Utilisation mémoire accrue (donnée + mémoire)
- Utilisation CPU accrue (récupération et accès d'une adresse)

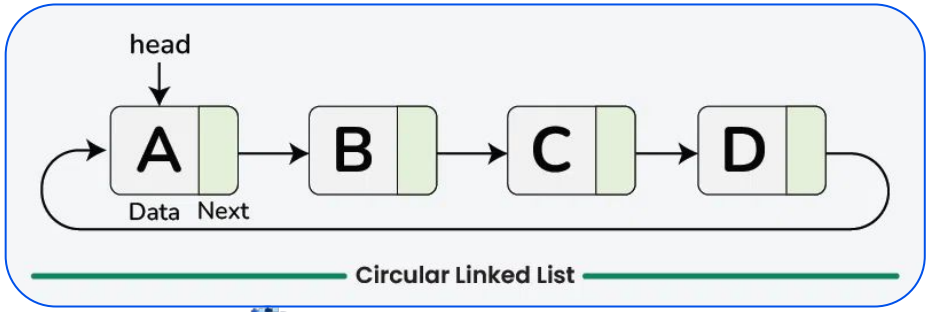


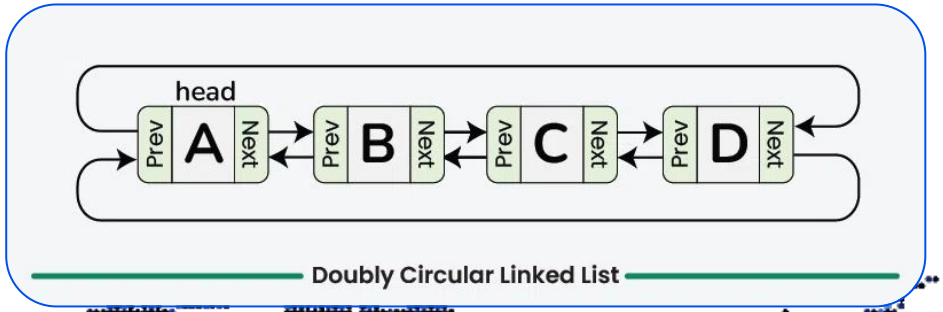


Liste chaînée
Quelques variantes









Quel est le point commun entre toutes ces images ?

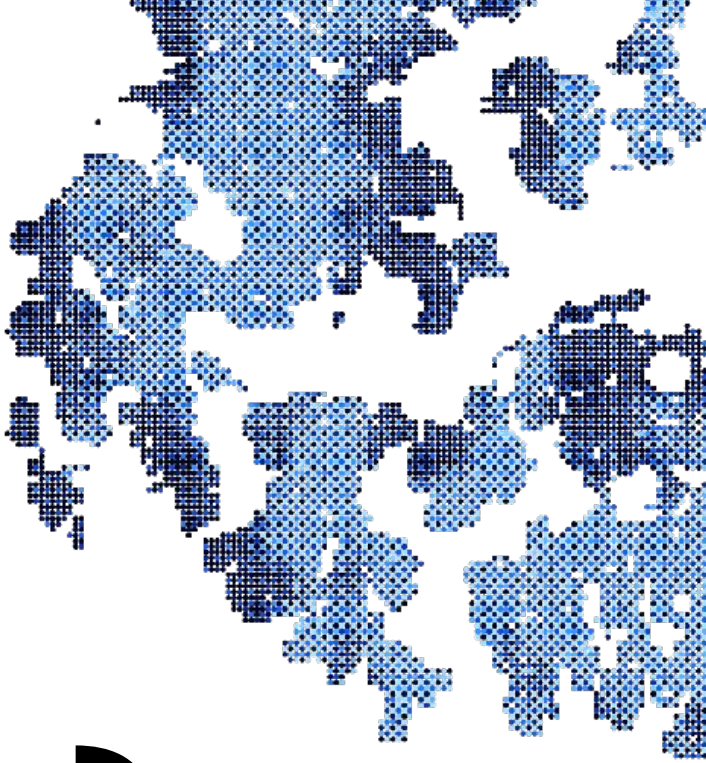






Ce sont toutes des piles !

INSAlgo



Stack (pile)

Stack (pile)

Une pile a 2 opérations principales :

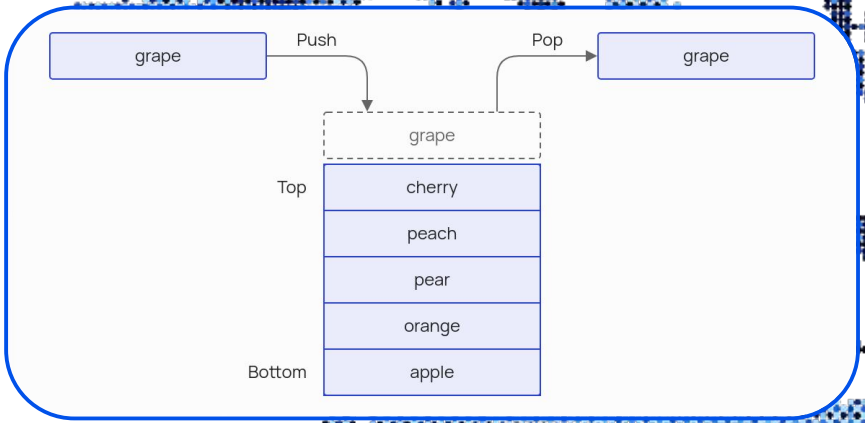
- **Push** pour ajouter un élément au dessus
- **Pop** pour enlever l'élément le plus récemment ajouté

Une pile est une **LIFO** : **L**ast **I**n **F**irst **O**ut

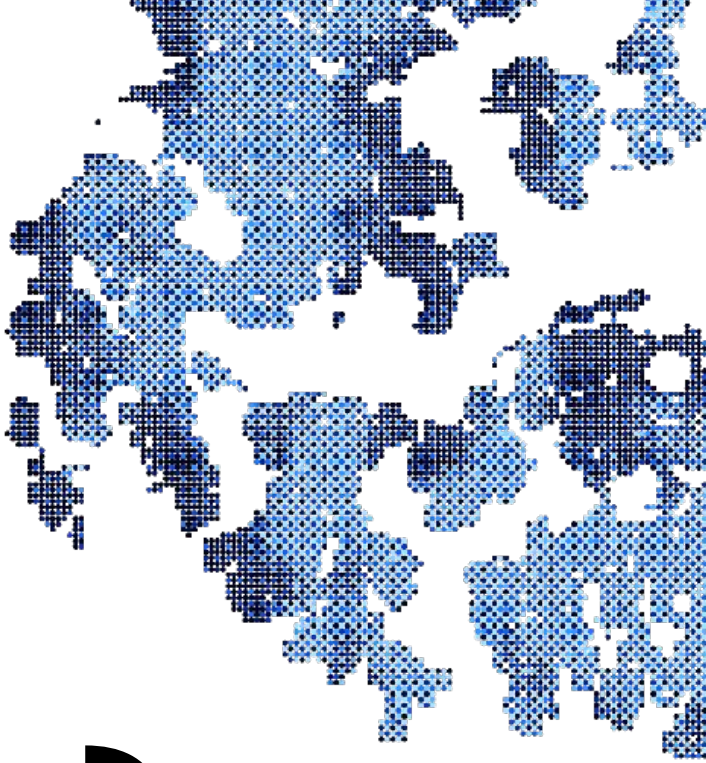
On peut l'implémenter en python avec une liste :

- list.append() → Push
- list.pop() → Pop

→ Pour une meilleure performance, il vaut mieux utiliser *collection.deque*



INSAlgo



Queue (file)

Queue (file)

Une file a 2 opérations principales :

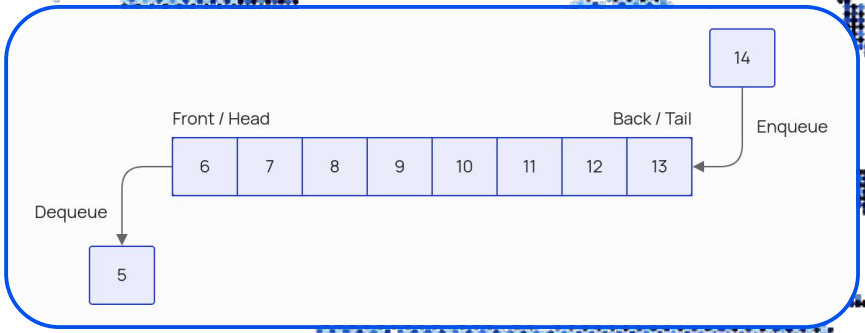
- **Enqueue** pour ajouter un élément au dessus
- **Dequeue** pour enlever l'élément au début de la file

Une file est une **FIFO** : **F**irst **I**n **F**irst **O**ut

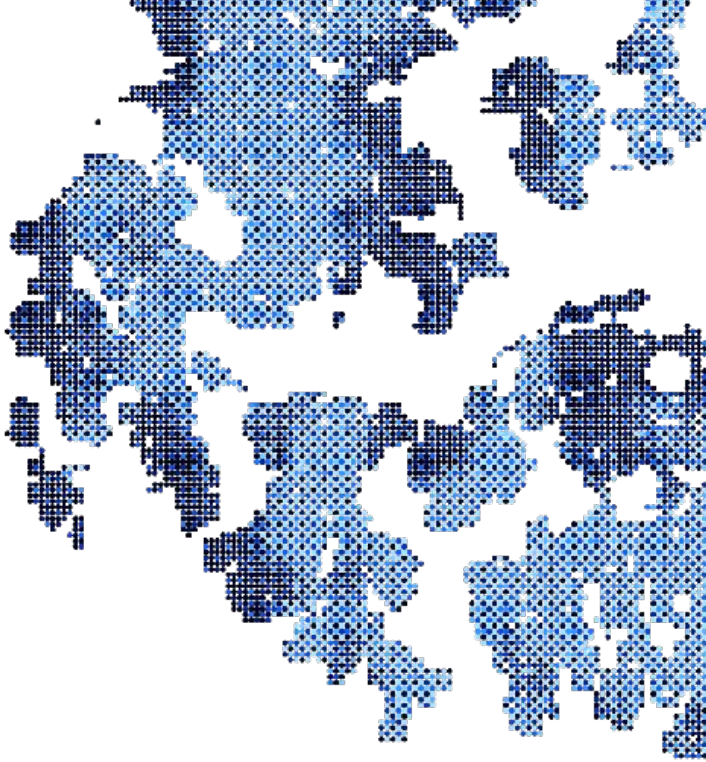
On peut l'implémenter en python avec une liste :

- list.append() → Enqueue
- del list[0] → Dequeue

→ Pour une meilleure performance, il vaut mieux utiliser *collection.deque* (avec *popleft()*)



INSAlgo



Tables de hash et dictionnaires

Ne pas confondre table de hash et dictionnaire !

Table de hash = le concept, la structure de données

Dictionnaire = l’implémentation en python



Dictionnaire

Objectif :

- Lier une **clé** à une **valeur**
- Retrouver la valeur grâce à la clé en O(1)

→ Non-ordonné (contrairement au dictionnaire en tant qu'ouvrage/outil)

Petite question : quelle est la complexité de retrouver un mot dans le dictionnaire ? (en supposant que vous connaissez l'ordre alphabétique :p)



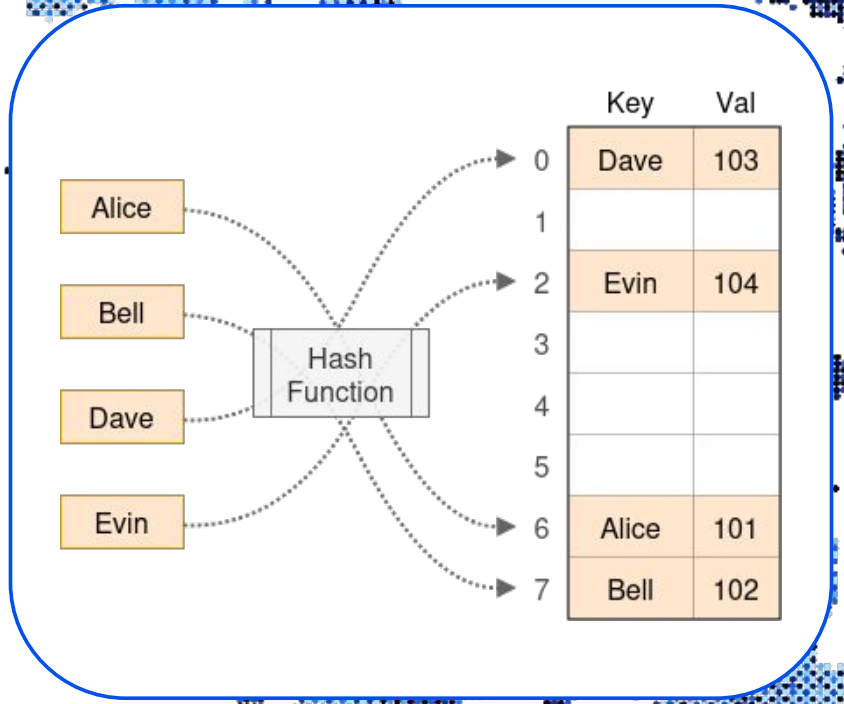


Dictionnaire et table de hash

La structure sous-jacente d'un dictionnaire est une **table de hash** :

- Une fonction de hash qui convertit la clé en un **index**
- Le duo clé/valeur est stocké dans une structure de donnée à cet **index**

Deux clés différentes peuvent obtenir le même index en sortie de la fonction de hash
→ À quoi peut ressembler cette fonction de hash ?
→ Comment gérer cette collision ?



Dictionnaire et table de hash

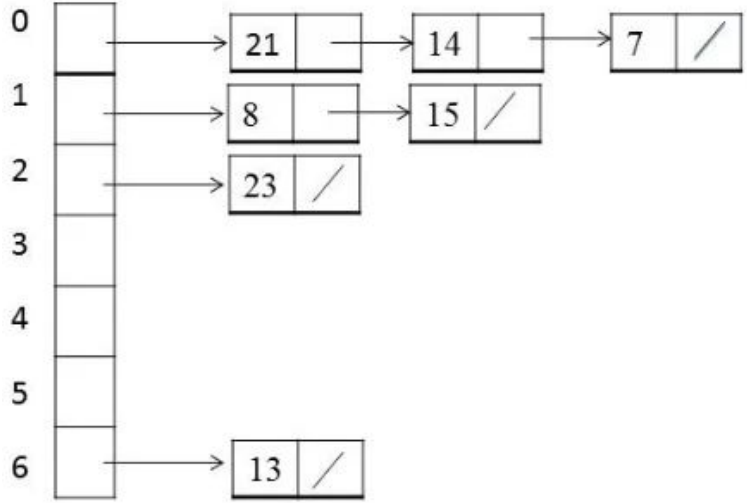
Gestion de la collision - Seperate chaining

→ Les couples (clé, valeur) ayant le même index de hash sont stockées **dans la même structure de données**

Ces structures de données sont gardées petites pour garantir l'accès à une valeur en O(1)

Structure de données fréquements utilisées :

- Liste chaînée (linked list)
- Arbre (tree)

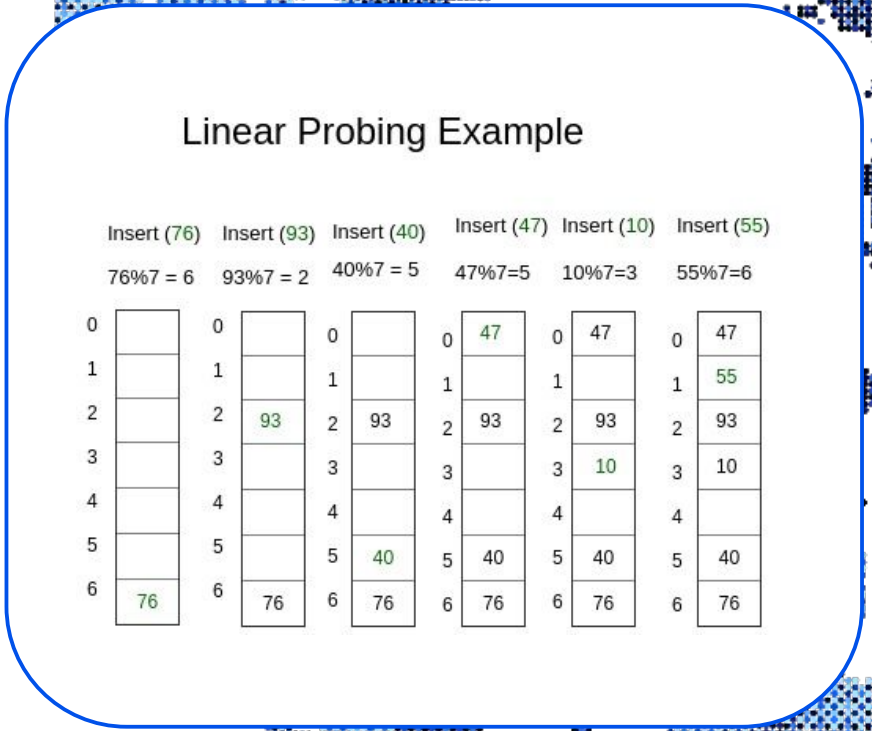


Dictionnaire et table de hash
Gestion de la collision - Probing

→ Les couples (clé, valeur) ayant le même index de hash sont stockées **dans un autre index**, pas de structure de données utilisée

Deux exemples de probing :

- **Linear probing** : on ajoute au prochain index
- **Random probing** : on suit une séquence aléatoire issue du hash



Dictionnaire et table de hash

Quand l'utiliser :

- Dès lors que l'on associe une valeur à une clé
- En Python, le type est natif et déjà optimisé, ce qui en fait un outil très puissant

Quand ne pas l'utiliser

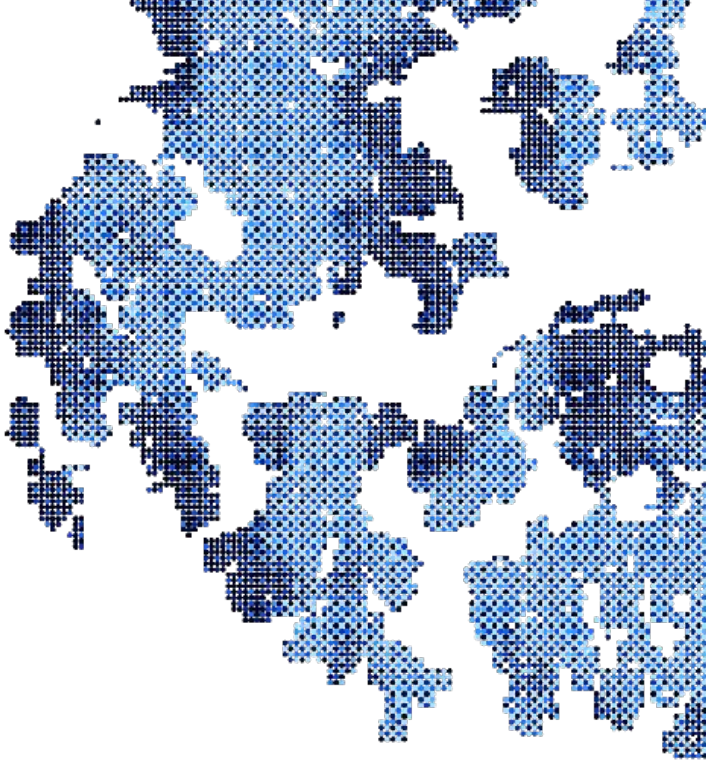
- Lorsque les clés sont 0, 1, ..., n → vous connaissez mieux !



Dictionnaire - Cheat sheet

Opération	**Python**	**Complexité**
Créer un dictionnaire	d = dict() ou d = {}	O(1)
Accéder à un élément	d[key]	~O(1)
Ajouter un élément	d[key] = value	~O(1)
Vérifier si une clé est dans le dictionnaire	key in d → booléen	~O(1)
Enlever un couple (clé, valeur)	del d[key]	~O(1)

INSAlgo



Set (ensembles)

Set (ensembles)

Objectif :

- Stocker des valeurs **uniques**
- Savoir rapidement si un élément est dans l'ensemble ou non

Deux implémentations typiques :

- Avec un **arbre** → ordonné
- Avec une **table de hash** → plus rapide mais non ordonné (ce sont des tables de hash mais sans valeur)





Set (ensembles) - Cheat sheet

Opération	Python	Complexité
Créer un ensemble	S = set()	O(1)
Accéder à un élément spécifique	IMPOSSIBLE	
Ajouter un élément	s.add(element)	~O(1)
Vérifier si un élément est dans l'ensemble	Element in d → booléen	~O(1)
Enlever un élément	s.remove(element)	~O(1)

Set (ensembles) - Opérations arithmétiques

Opération	Python	Complexité
Vérifie si s1 est un [proper] ***subset*** de s2 → Vrai si s2 contient tous les éléments de s1 [proper] → et au moins un autre élément	s1 <= s2 et s1 < s2	O(n1)
Vérifie si s1 est un [proper] ***superset*** de s2 → Vrai si s1 contient tous les éléments de s2 [proper] → et au moins un autre élément	s1 >= s2 et s1 > s2	O(n2)
Union des ensembles s1, s2, ..., sk	s1 | s2 | ... | sk	O(n1, n2, ..., nk)
Intersection des ensembles s1, s2, ..., sk	s1 & s2 & ... & sk	O(min(n1, n2, ..., nk))
Tous les éléments de s1 qui ne sont pas dans s1	s1 - s2	O(n1)
Tous les éléments qui sont dans s1 ou s2 mais pas les deux en même temps	s1 ^ s2	O(n1 + n2)

Set (ensembles)

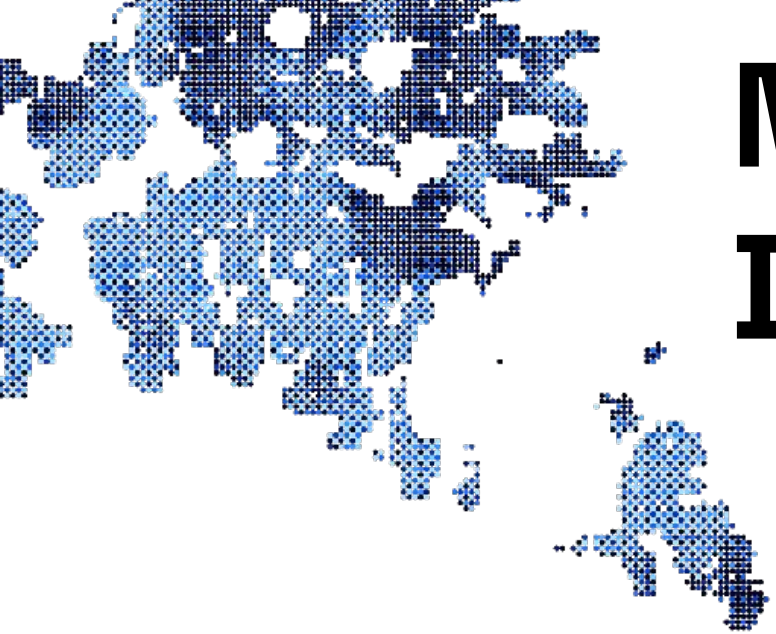
Quand l’utiliser :

- Pour marquer des valeurs déjà traitées
- Pour garder une collection d’éléments uniques non-ordonnées

Quand ne pas l’utiliser

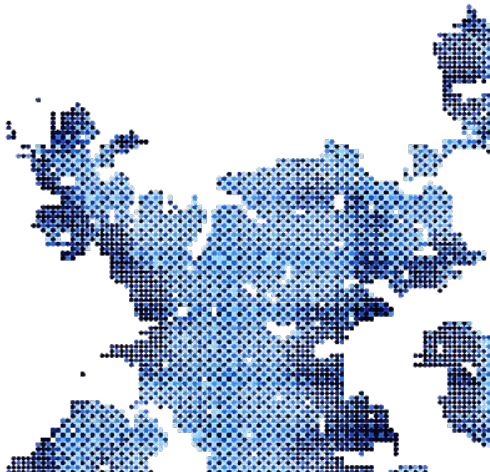
- Quand vous avez besoin de plusieurs fois le même élément…





Merci !
Des questions ?

Passons aux exercices :D



Crédits (contenu et slides) : Arthur Tondereau, Onyr (Florian Rascoussier), Nicolas Foussard